VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Software Engineering

Extended-Course Project Report

Mini Software Engineering Project - Smart Event Reminder System (SERS)

Class A01 – Group CNPM_TN1 – Semester 252

Advisor(s):	Nguyễn Thành Công	
Student(s):	Nguyễn Hữu Thịnh	2313292
	Nguyễn Chí Thanh	2313078
	Nguyễn Việt Hoàng	2311066
	Lê Đức Tài	2312995
	Mai Xuân Nhựt	2312549

HO CHI MINH CITY, MAY 2026

Contents

1 Requirement Engeneering

1.1 Stakeholder Identification

The initial phase involves identifying all entities that interact with, benefit from, or govern the software application.

- **User:** The primary human consumer of the application who interfaces with the graphical user interface to manage calendar events.
- **System Administrator:** An operational overseer equipped with elevated privileges, responsible for managing accounts, monitoring system health, and resolving data conflicts.
- **Notification Service:** An internal programmatic actor or external third-party integration (like an SMTP server or SMS gateway) that routes and delivers asynchronous reminders.
- **Project Engineering Team:** The five-member technical unit responsible for designing, implementing, verifying, and maintaining the system architecture.
- **Instructor:** The governing authority responsible for evaluating the fidelity of the software engineering process and documentation.

1.2 Functional Requirement(FRs)

Functional requirements define the explicit behaviors and data processing operations the system must execute

- **FR-01: Event Lifecycle Management**: The system must enable an authenticated user to create, read, update, and delete (CRUD) event entries, enforcing a valid future date and time parameter.
- **FR-02: Metadata Association**: The system must allow users to append optional metadata, specifically expanded textual descriptions and geographical location data.
- **FR-03: Alert Schedule Calculation**: The system must calculate a specific notification trigger timestamp based on the user-defined reminder interval subtracted from the primary event timestamp.
- **FR-04: Asynchronous Alert Execution**: The system must asynchronously dispatch the reminder via the Notification Service at the calculated trigger timestamp without blocking the main application thread.

- **FR-05: User Authentication Protocol**: The system must authenticate users via secure credentials to ensure event data is strongly siloed.
- **FR-06: Event Sharing Interface**: The system should provide a mechanism allowing a user to share event details or digitally invite other registered users.

1.3 Non-Functional Requirements (NFRs)

Non-functional requirements act as systemic constraints, dictating how the system must behave under various operational loads

- **NFR-01: Performance and Latency**: The graphical user interface must respond to synchronous user inputs (e.g., saving a new event) in under 1,000 milliseconds to prevent user frustration.
- **NFR-02: System Scalability**: The asynchronous notification engine must be linearly scalable, capable of processing up to 50,000 asynchronous notifications per day without degrading the primary web server's performance.
- **NFR-03: Reliability and Availability**: The application infrastructure must maintain an operational uptime of 99.9

1.4 User Stories & Acceptance Criteria

User stories utilize a standard human-centric narrative format: "As a [type of user], I want [objective] so that [benefit]". Each story is accompanied by explicit Acceptance Criteria to define the conditions for completion.

- **US-01: Create Event**
  - **User Story:** As a user, I want to create a new event with a title, date, and time, so that I can maintain an accurate digital record of my upcoming appointments.
  - **Acceptance Criteria:**
    1. The input form cryptographically and logically validates the title, date, and time.
    2. The event object successfully persists to the database.
    3. The graphical interface displays a success confirmation modal.
- **US-02: Schedule Reminder**

  - **User Story:** As a user, I want to set a specific reminder time interval prior to an event, so that I am alerted automatically before the occurrence begins.
  - **Acceptance Criteria:**
    1. The system interface accepts numerical time intervals.
    2. The backend mathematical engine calculates the exact trigger time.
    3. The background scheduler successfully registers the task in the message broker queue.

- **US-03: Edit Event**
  - **User Story:** As a user, I want to edit an existing event's specific details, so that my schedule remains accurate if locations or times change unexpectedly.
  - **Acceptance Criteria:**
    1. The user can retrieve and populate an existing event into the input form.
    2. Modifications overwrite the existing database record.
    3. Any previously scheduled reminder triggers associated with the old time are successfully revoked and replaced.

- **US-04: Delete Event**
  - **User Story:** As a user, I want to delete a canceled event entirely, so that my application dashboard remains uncluttered and accurate.
  - **Acceptance Criteria:**
    1. The deletion command removes the event from the client view.
    2. The corresponding database record is permanently dropped.
    3. All pending notification tasks associated with the event ID are aborted in the background queue.

- **US-05: View System Logs**
  - **User Story:** As an administrator, I want to view system logs for failed notification deliveries, so that I can troubleshoot infrastructure or SMTP delivery issues proactively.
  - **Acceptance Criteria:**

1. The secure administrator dashboard queries and displays error logs.
2. The log entries explicitly indicate the failed event ID, the target user, and the failure timestamp.

1.5 MoSCoW Prioritization Framework

To prevent scope creep and ensure the delivery of a Minimum Viable Product (MVP), the team utilizes the MoSCoW prioritization framework.

Must Have: Features vital to the system's existence. This includes basic event creation, event deletion, logical reminder scheduling, and the asynchronous dispatch of the notification. Without these, SERS fails to function as a reminder application.

Should Have: Important features that add significant value but are not strictly existential. This includes event editing, recurring event configurations, and the ability to set multiple reminder intervals (e.g., an alert one hour prior, and another ten minutes prior).

Could Have: Desirable enhancements that will only be addressed if sprint capacity permits. This encompasses the optional functionality of inviting other users, sharing events via direct email links, and automated workflow suggestions based on calendar availability.

Won't Have (This Iteration): Complex functionalities explicitly deferred to future releases. This includes integration with third-party proprietary calendars (e.g., Google Calendar APIs), complex geographical routing for travel time alerts, and AI-driven predictive scheduling.

1.6 Requirements Traceability Matrix (RTM)

The Requirements Traceability Matrix (RTM) mathematically proves that every functional requirement is tethered to a specific user story, which in turn maps to an architectural component and a definitive test case. This linkage guarantees that:

- No orphaned code is written without a corresponding requirement.
- No requirement is left untested.
- Every architectural component has a clear functional justification.

Functional Req.	Correlated User Story	Primary Architectural Component	Assigned Test ID
FR-01 (Event Lifecycle)	US-01, US-03, US-04	`EventController`, `EventModel`	T1, T2
FR-02 (Metadata)	US-01	`EventModel`, `MetadataField`	T1
FR-03 (Alert Calculation)	US-02	`SchedulerService`, `TaskQueue`	T3, T4
FR-04 (Async Dispatch)	US-02	`NotificationWorker`, `SMTP Client`	T3, T5
FR-05 (Authentication)	US-05	`AuthController`, `UserModel`	T6
FR-06 (Event Sharing)	US-05	`SharingService`, `InviteModel`	T7

Table 1.1: Requirements Traceability Matrix for SERS Project

2 Process Model and Planning

- **Process Model Selection**

  For the Smart Event Reminder System (SERS), the team adopts the **Agile Scrum** model.

  Scrum is selected because the project has a short duration (approximately two weeks), involves evolving requirements, and requires continuous integration between multiple components such as the user interface, backend logic, and asynchronous notification system. Compared to the Waterfall model, Scrum allows the team to quickly adapt to unexpected issues (e.g., scheduling errors or integration problems). Compared to the Incre-

mental model, Scrum provides more structured collaboration through regular meetings and feedback loops.

Therefore, Scrum is the most suitable model for ensuring flexibility, collaboration, and timely delivery.

- **Product Backlog**

  The Product Backlog contains all features required for the system, prioritized based on importance:

Table 2.1: Product Backlog

ID	Backlog Item	Priority
PB1	Create event (title, date, time)	Must
PB2	Edit event	Must
PB3	Delete event	Must
PB4	Schedule reminder	Must
PB5	Send notification	Should
PB6	User authentication	Should
PB7	Share event	Could

- **Effort Estimation (Planning Poker)**

  The team uses Planning Poker with Fibonacci scale (0, 1, 2, 3, 5, 8, 13, 21) to estimate effort:

Table 2.2: Story Points Estimates

Backlog Item	Story Points
Database schema	3
CRUD API	8
Basic UI	10
Schedule reminder	8
Send notification	5
Authentication	8
Share event	3

- **Sprint Planning**

  The project is divided into two sprints (1 week each), ensuring incremental delivery.

Sprint 1: Core System Setup (21 Story Points) The team commits to the following backlog items:

- Initialize database schema (3 points)
- Implement CRUD API for events (8 points)
- Develop basic UI dashboard (10 points)

Total: 21 story points

This sprint focuses on delivering a functional core system (Minimum Viable Product – MVP), allowing users to create, view, edit, and delete events.

—

Sprint 2: Advanced Features (Remaining Story Points) The second sprint focuses on extending system functionality:

- Reminder scheduling (8 points)
- Notification system (5 points)
- User authentication (8 points)
- Event sharing (3 points, optional)

This sprint enhances the system by adding automation, security, and additional user features.

—

This two-sprint structure ensures that a working system is delivered early (Sprint 1), while advanced capabilities are incrementally added in Sprint 2.

- **Team Task Distribution**
  To support parallel development, tasks are distributed among five team members:
  - Member A: Backend (CRUD API, database)
  - Member B: Frontend (UI dashboard)
  - Member C: Asynchronous system (reminder & notification)
  - Member D: Testing and QA
  - Member E: Documentation and integration

- **Sprint Day (Simulated 10-Day Cycle)**

Table 2.3: Sprint Day (Simulated 10-Day Cycle)

Day	Remaining Story Points	Progress Narrative
Day 1	21 Points	The sprint begins. All parallel workstreams initiate development on their assigned components.
Day 3	18 Points	The database schema architecture is finalized and committed. Points are burned down successfully.
Day 6	10 Points	The RESTful API backend logic is completed. The team maintains pace with the ideal burndown trajectory.
Day 8	8 Points	A slight plateau occurs. The frontend integration encounters cross-origin resource sharing (CORS) errors, delaying the dashboard completion.
Day 10	0 Points	Integration issues are resolved. The dashboard is completed, and all sprint objectives are achieved prior to the sprint review.

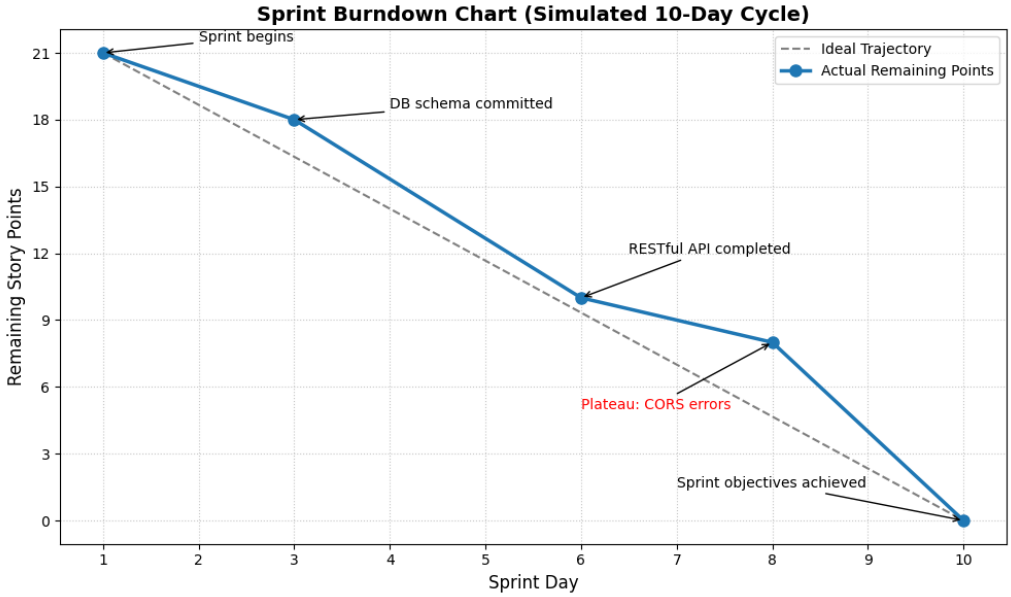

Figure 2.1: Burndown chart (Simulated 10-Day Cycle)

- **Summary**

  Using Scrum enables the team to manage work effectively, adapt to technical challenges,

and deliver the system incrementally. The use of backlog prioritization, effort estimation, and sprint planning ensures that the project remains organized and achievable within the given timeframe.

3 Architectural Design

Translating the abstract UML models into a functioning codebase requires the selection of an overarching architectural pattern. The SERS application utilizes the Model-View-Controller (MVC) paradigm, heavily augmented by an asynchronous message-brokering layer. The MVC pattern enforces a rigorous separation of concerns. The Model layer encapsulates all business logic, data validation, and database interactions. The View layer is strictly confined to presentation logistics, rendering the user interface and accepting input. The Controller acts as the central orchestrator, receiving HTTP requests from the View, commanding the Model to update its state, and managing the application flow.

However, standard MVC is fundamentally synchronous; a Controller typically blocks the server thread until a task completes. For a reminder system, holding an HTTP connection open for three days until a scheduled event occurs is architecturally catastrophic. Therefore, the technology stack is designed to offload temporal logic. The backend Controller and Model are constructed using Python and the Flask framework. Data is persisted using an SQLite database (capable of scaling to PostgreSQL) mapped via SQLAlchemy. Crucially, the asynchronous layer utilizes Celery as the task queue manager and Redis as the in-memory message broker. When the Flask Controller receives a scheduling request, it pushes the task parameters into the Redis broker and immediately responds to the user. Independent Celery worker processes continuously monitor the Redis broker, executing the tasks only when their specific temporal conditions are met.

4 Implementation & Code Quality

The theoretical architecture is demonstrated in the following Python implementation snippet, which encapsulates the Controller logic and the Celery background worker.

```
1 import os
2 import datetime
3 from celery import Celery
4 from sqlalchemy import create_engine, Column, Integer, String, DateTime
```

```
from sqlalchemy.orm import declarative_base, sessionmaker

# Architecture Initialization: Message Broker and Database Config
DATABASE_URI = os.getenv('PROD_DATABASE_URL', 'sqlite:///sers_local.db')
REDIS_BROKER = os.getenv('CELERY_BROKER_URL', 'redis://localhost:6379/0')

app_celery = Celery('sers_async_worker', broker=REDIS_BROKER)
app_celery.conf.timezone = 'UTC'

engine = create_engine(DATABASE_URI)
Base = declarative_base()
Session = sessionmaker(bind=engine)

# Model Layer: Data encapsulation mapping to the UML Class Diagram
class EventEntity(Base):
    __tablename__ = 'scheduled_events'
    id = Column(Integer, primary_key=True)
    title = Column(String(150), nullable=False)
    event_timestamp = Column(DateTime, nullable=False)
    target_email = Column(String(150), nullable=False)
    async_task_id = Column(String(100)) # Stores Celery ID for revocation

Base.metadata.create_all(engine)

# Asynchronous Worker Layer: Executed completely independent of the web server
@app_celery.task(name='notification.dispatch', bind=True)
def dispatch_reminder_alert(self, entity_id, target_email, event_title):
    """
    Background worker simulating external SMTP connection.
    This function blocks its own thread, but does not block the web server.
    """
    try:
        alert_payload = f"Automated Reminder: '{event_title}' is approaching."

        # System logging to verify successful delivery for administrative review
        log_entry = f"[{datetime.datetime.utcnow().isoformat()}] SUCCESS -> {target_email} | Payload:
    ↪ {alert_payload}\n"
        with open("system_delivery_logs.txt", "a") as file_handle:
            file_handle.write(log_entry)

        return {"execution_status": "success", "entity_id": entity_id}
    except Exception as runtime_error:
        return {"execution_status": "failed", "error_trace": str(runtime_error)}

# Controller Layer: Orchestrates Model updates and Broker dispatch
class SersController:
    def __init__(self):
        self.db_session = Session()

    def schedule_new_event(self, title, event_timestamp, target_email, offset_minutes):
        """
        Validates input, persists the Model, calculates temporal offsets,
        and pushes the payload to the Redis broker.
        """
        # Primary logical validation
        if not title or not target_email:
            raise ValueError("Data Validation Failure: Title and Email are mandatory parameters.")

        current_system_time = datetime.datetime.utcnow()
        if event_timestamp <= current_system_time:
```

```
            raise ValueError("Temporal Logic Failure: Scheduled events must occur in the future.")

        # 1. Persist the Model state
        new_event = EventEntity(
            title=title,
            event_timestamp=event_timestamp,
            target_email=target_email
        )
        self.db_session.add(new_event)
        self.db_session.commit()

        # 2. Calculate precise temporal trigger offset
        calculated_trigger = event_timestamp - datetime.timedelta(minutes=offset_minutes)
        if calculated_trigger < current_system_time:
            # Fallback mechanism if offset pushes trigger into the past
            calculated_trigger = current_system_time + datetime.timedelta(seconds=5)

        # 3. Dispatch the payload to the asynchronous broker
        dispatched_task = dispatch_reminder_alert.apply_async(
            args=[new_event.id, new_event.target_email, new_event.title],
            eta=calculated_trigger
        )

        # 4. Update Model with task identifier to allow future revocation
        new_event.async_task_id = dispatched_task.id
        self.db_session.commit()

        return new_event.id

    def revoke_existing_event(self, entity_id):
        """
        Removes the Model entity and aggressively terminates the pending asynchronous task.
        """
        target_event = self.db_session.query(EventEntity).filter_by(id=entity_id).first()
        if target_event:
            if target_event.async_task_id:
                # Intercept the message broker to prevent misfire of canceled events
                app_celery.control.revoke(target_event.async_task_id, terminate=True)
            self.db_session.delete(target_event)
            self.db_session.commit()
            return True
        return False
```

The inspection process focuses on identifying Code Smells—structural indicators of poor design that, while not necessarily preventing the program from functioning, significantly increase technical debt and the probability of future catastrophic failures.

Table 4.1: Identified Code Smells and Refactoring Strategies

Identified Code Smell Category	**Technical Description and Observation**	**Proposed Refactoring Strategy**
Bloater: Long Method	The `schedule_new_event` method handles input validation, database instantiation, complex mathematical time calculations, and asynchronous message brokering simultaneously. It violates the Single Responsibility Principle by attempting to manage too many disparate layers of logic.3	Extract Method: The mathematical calculation of the temporal offset and the actual dispatching to the Celery broker should be abstracted into a private helper function (e.g., `_dispatch_to_broker()`). This isolates the temporal logic, making the primary method vastly more readable and testable.3
Object-Orientation Abuser: Primitive Obsession	Passing discrete primitive variables (`title`, `event_timestamp`, `target_email`, `offset_minutes`) linearly through multiple architectural layers heavily clutters the method signatures, resulting in Long Parameter Lists and code duplication.49	Introduce Parameter Object: The isolated primitives should be grouped into a single cohesive Data Transfer Object (DTO) or a formalized validation model (such as a Pydantic schema). Passing a single `EventPayload` object ensures strict type safety across boundaries.3

Identified Code Smell Category	**Technical Description and Observation**	**Proposed Refactoring Strategy**
Concurrency Anti-Pattern: Unsafe Logging	Utilizing `open("system_delivery_logs.txt", "a")` directly within the asynchronous worker is highly dangerous. If multiple Celery workers attempt to append to the same text file simultaneously, it will result in file lock collision errors and lost data.	Implementation Refactor: Remove raw file I/O operations from background workers. Replace with a thread-safe Python logging module configuration or route logs through a secondary stream processing broker.